ADITYA UNIVERSITY

UNIT-5: Multiprocessors & Pipelining

UNIT-V: Multiprocessors & Pipelining

PART-A: Multi-Processors

- Characteristics of Multiprocessors
- Interconnection Structures
- Inter Processor Arbitration

PART-b: Pipelining

- Parallel Processing
- Pipelining
- Instruction Pipeline
- RISC Pipeline
- Array Processor

ADITYA UNIVERSITY

UNIT-V

PART-A: Multi-Processors

- Characteristics of Multiprocessors
- Interconnection Structures
- Inter Processor Arbitration

ADITYA UNIVERSITY

Characteristics of Multiprocessors

❑ **Basic Structure**

- A multiprocessor system consists of two or more CPUs connected with memory and I/O devices.
- It is controlled by a **single operating system**.

❑ **Parallel Processing Capability**

- Supports concurrent (parallel) execution of multiple tasks.
- Follows MIMD (Multiple Instruction, Multiple Data) organization.

❑ **Performance Improvement**

- Provides high system performance due to parallel execution.
- Multiple jobs or parts of a single job can run simultaneously.

❑ **Reliability (Fault Tolerance)**

- Improves system reliability.
- If one processor fails, others can continue execution.

❑ **Task Execution Methods**

- Multiple independent jobs can run in parallel.
- A single job can be divided into smaller parallel tasks.

❑ **Resource Sharing**

- Processors share memory and I/O resources efficiently.
- Enables better utilization of system resources.

❑ **Specialized Processing**

- Some processors may be designed for specific tasks to improve efficiency.

❑ **Memory Organization**

Tightly Coupled System

- Uses shared common memory.
- Provides faster communication between processors.

Loosely Coupled System

- Each processor has its own local memory.
- Communication is done through message passing.

ADITYA UNIVERSITY

Interconnection Structures

- A multiprocessor system consists of **CPUs, Input–Output Processors (IOPs), and a memory unit**, all interconnected using an interconnection network or interconnection structure.
- Types of Interconnection Structures
  - **Time-Shared Common Bus:** All processors share a common bus to access the main memory, but only one can use it at a time.
  - **Multiport Memory:** Memory has multiple ports so that more than one processor can access it at the same time.
  - **Crossbar Switch:** Each processor can be connected to any memory module through switching paths.
  - **Multistage Switching Network:** Processors and memory are connected through multiple stages of switches.
  - **Hypercube System:** Processors are connected in a cube structure to allow communication between them.

Interconnection Structures

Time-Shared Common Bus

- Single bus connects all processors and memory as shown in Fig
- Only one processor can access bus at a time
- Conflicts resolved using priority schemes
- Simple and low cost but low speed
- Can be improved using multiple buses

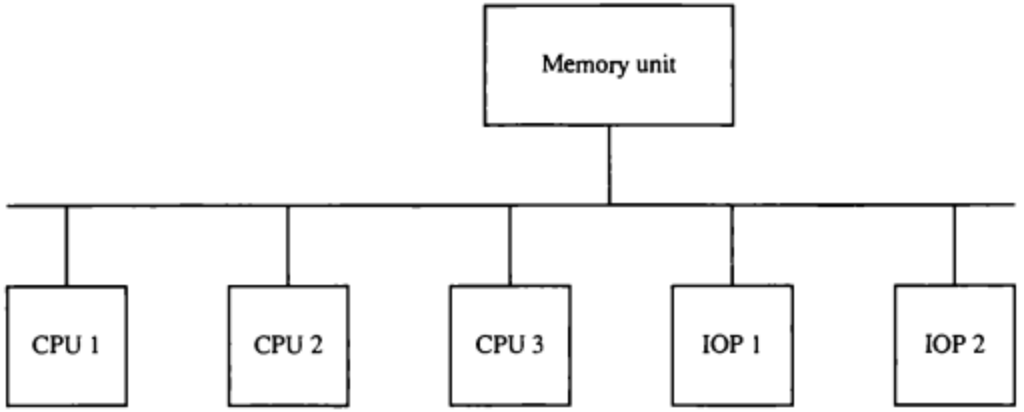


Fig: Time-Shared common bus organization

System Bus Structure

- Each processor has a local bus as shown in Fig.
- Connected to a common system bus
- Includes:
  CPU, IOP, local memory
- Shared memory connected to system bus
- Improves access speed

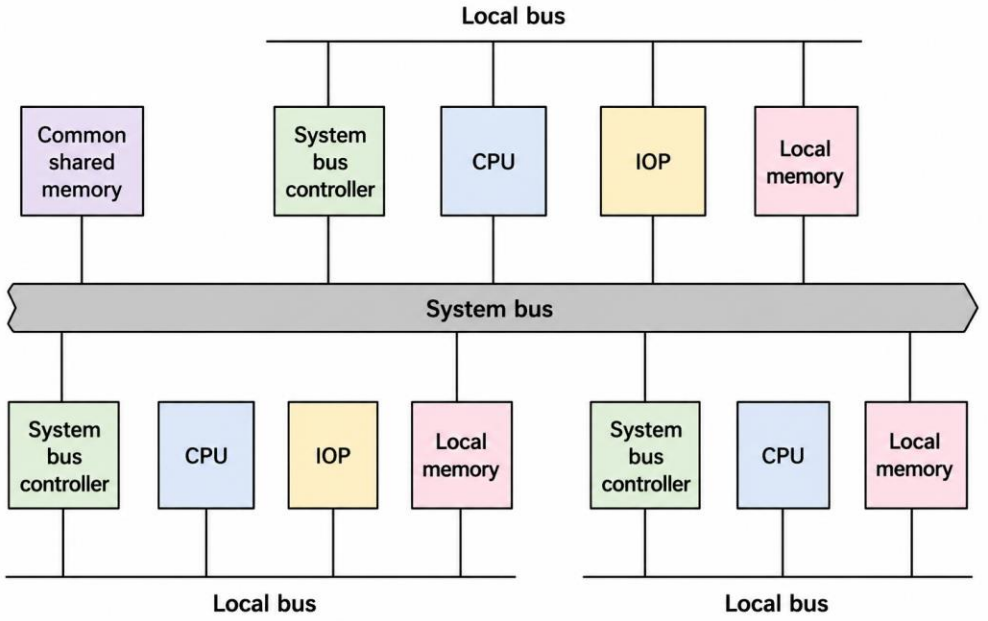


Fig: System bus Architecture for Multiprocessors

Interconnection Structures

Multiport Memory Connection

- Memory modules have multiple ports as shown in Fig.
- Each processor connects to all memory modules as shown in Fig. and allows simultaneous access
- Uses arbitration logic (Arbitration logic selects one based on priority)
- High transfer speed, Expensive and complex.

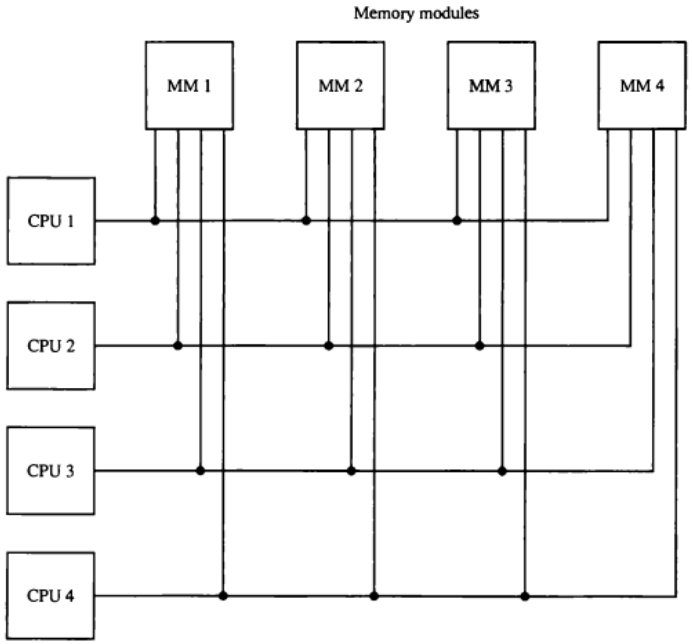


Fig: Multiport Memory Organization

Crossbar Switch

- Uses grid of switches (crosspoints)
- Connects any processor to any memory
- Supports parallel communication
- High performance, Very costly due to many switches

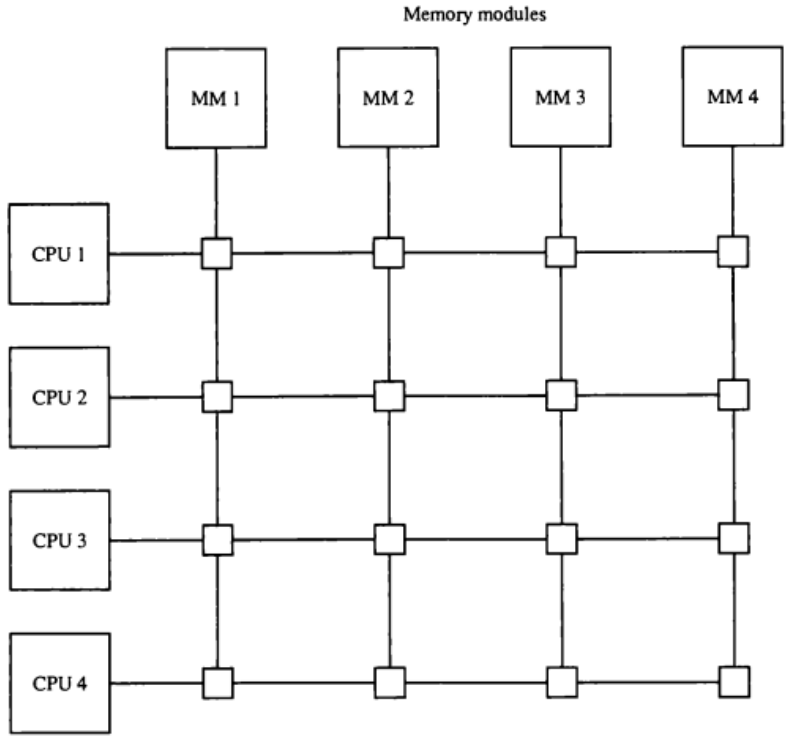


Fig: Crossbar Switch

Interconnection Structures

Multistage Switching Network

- Uses multiple stages of small switches (2×2)
- Reduces hardware compared to crossbar
- An 8 × 8 Omega switching network is shown in Fig. where multiple inputs are connected to outputs through several stages of 2×2 switches.
- Data is routed stage by stage based on the binary address of the destination.
- Cost-effective and may have blocking problem

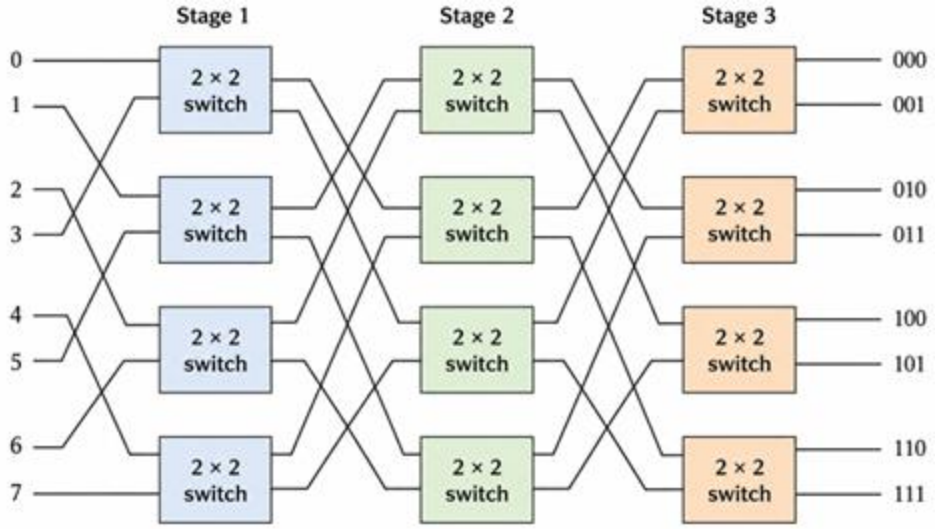


Fig: 8x8 Omega switching network

Hypercube System

- Processors connected in cube structure
- Number of processors = 2^n
- Each processor connects to n neighbors
- Uses binary addressing for routing
- Highly scalable and efficient

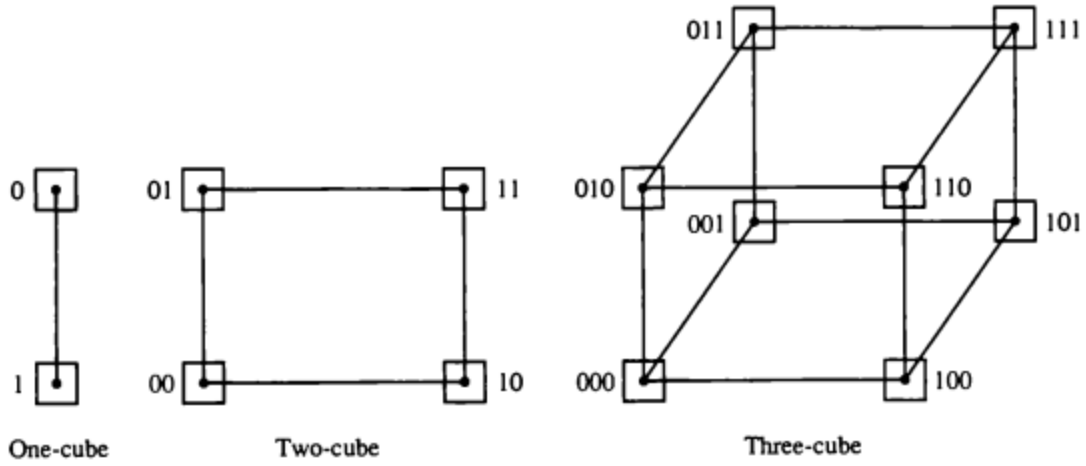


Fig: Hypercube structures for n=1,2,3

ADITYA UNIVERSITY

Inter Processor Arbitration

- Arbitration is the process of selecting one processor (based on priority) to use the shared bus when multiple processors request it at the same time.
- Multiprocessor systems use shared buses for communication.
- When multiple processors request the bus → conflict occurs.
- Arbitration decides which processor gets control of the bus.
- Implemented using hardware logic in the system bus controller.

Arbitration Techniques

1. Serial Arbitration (Daisy Chain)
2. Parallel Arbitration

Dynamic Arbitration Algorithmes

1. Time Slice: Each processor gets bus for a fixed time
2. Polling: Bus controller checks processors one by one
3. Least Recently Used (LRU): Gives priority to processor not used for longest time
4. FIFO (First-In First-Out): Requests served in order of arrival
5. Rotating Daisy Chain: Priority changes after each bus access

Serial Arbitration (Daisy Chain)

- Processors are connected in a chain of arbiters.
- The first processor has highest priority, last has lowest.
- Bus request signal passes sequentially from one arbiter to next.
- The first requesting processor gets control of the bus.
- The bus busy line indicates whether the bus is in use.
- It uses fixed priority, so lower-priority processors may wait longer.

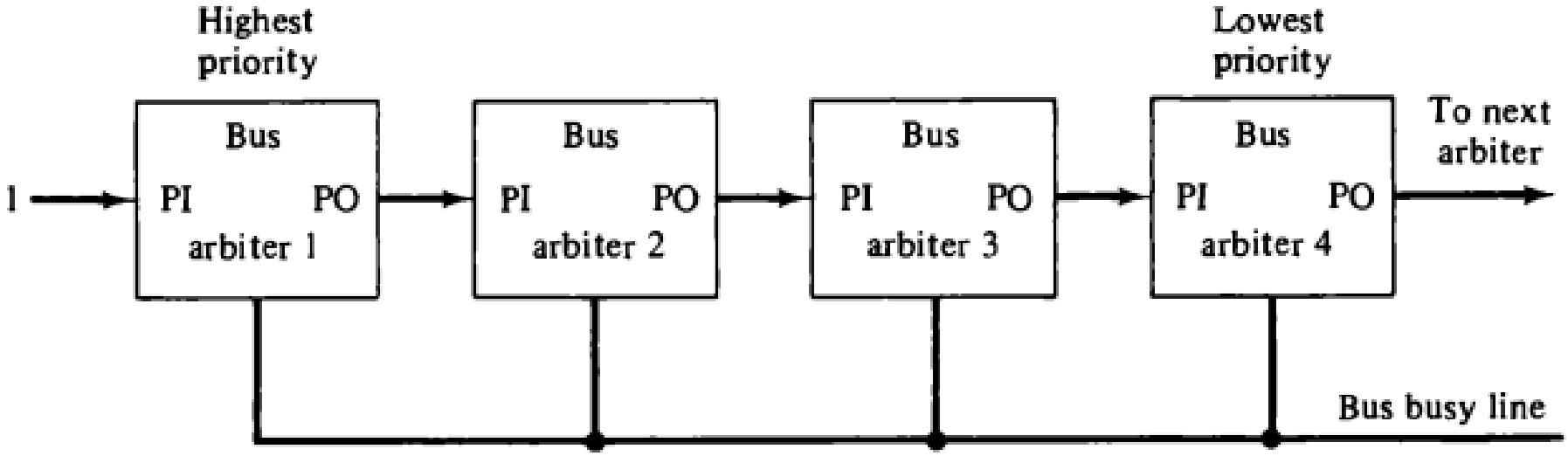


Fig: Serial Arbitration (Daisy Chain)

Parallel Arbitration

- The diagram shows a parallel arbitration system using a priority encoder and decoder.
- Each processor sends a bus request (Req) signal to the arbitration logic.
- All requests are given to a 4×2 priority encoder, which selects the highest priority request.
- The encoder output is converted by a 2×4 decoder to generate the acknowledge (Ack) signal.
- The selected processor receives the Ack signal and gets control of the bus.
- The bus busy line indicates whether the bus is currently in use.
- This method allows faster decision making compared to serial arbitration.
- However, it requires more hardware and complexity.

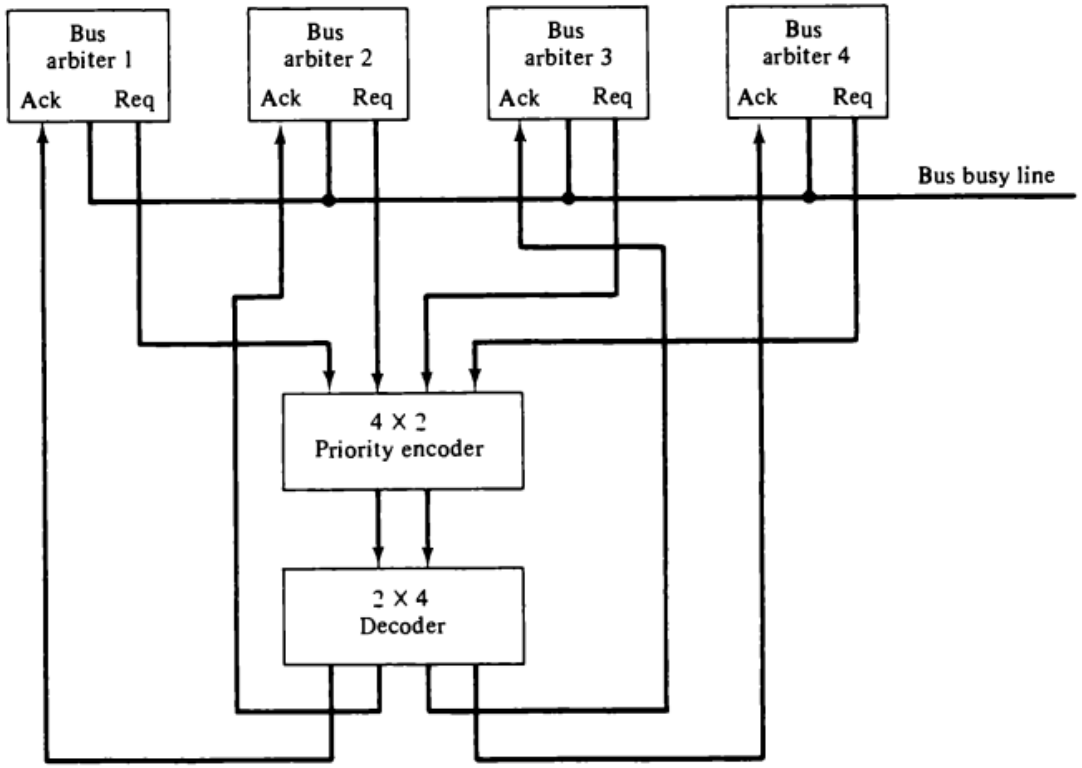


Fig: Parallel Arbitration

ADITYA UNIVERSITY

UNIT-V
PART-B: PIPELINING

- Parallel Processing
- Pipelining
- Instruction Pipeline
- RISC Pipeline
- Array Processor

Parallel Processing

- Parallel processing means performing multiple operations simultaneously using more than one processing units to improve the speed and throughput.
- **Types of Parallelism:**
  - **SISD (Single Instruction, Single Data):**

  Only one instruction works on one data at a time.

  This is normal serial processing (one task at a time).
  - **SIMD (Single Instruction, Multiple Data):**

  Same instruction is applied to many data values at the same time.

  Used in image processing, graphics, etc.
  - **MISD (Multiple Instruction, Single Data):**

  Different processors perform different instructions on one data.

  Used in multi-core computers.
  - **MIMD (Multiple Instruction, Multiple Data):**

  Different processors perform different instructions on multiple data.

  Used in multi-core computers.

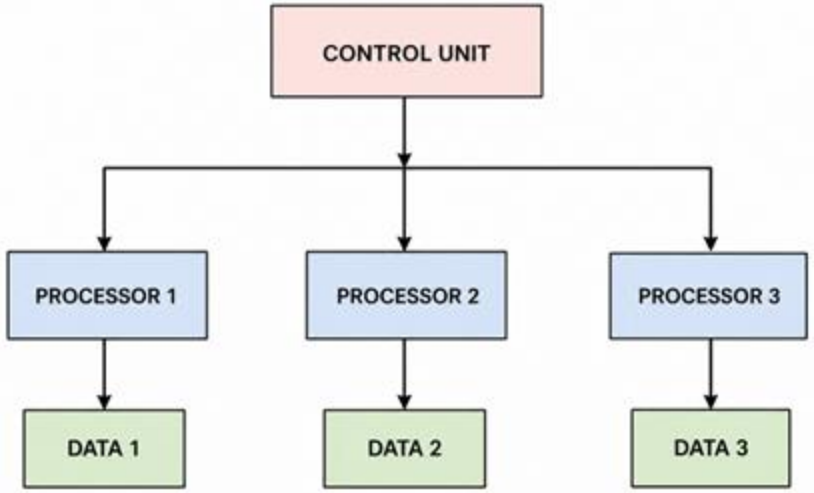


Fig: Block Diagram of Parallel Processing

- **Types of Parallel processing**
  - ✓ Pipeline Processing
  - ✓ Vector Processing
  - ✓ Array Processors

Pipelining with Example

- Pipelining is a technique used in digital computers to increase instruction throughput by dividing a task into a number of stages and executing multiple instructions simultaneously in overlapping fashion.
- Let us consider one example to demonstrate the pipelining concept. To perform multiply and addition on the data stream as $A_i \times B_i + C_i$ for $i = 1, 2, 3, ..., 7$
- The sub-operations performed in each segment of pipeline are follows as:

$R1 \leftarrow A_i$, $R2 \leftarrow B_i$	Input A_i and B_i
$R3 \leftarrow R1 * R2$, $R4 \leftarrow C_i$	Multiply and input C_i
$R5 \leftarrow R3 + R4$	Add C_i to product

TABLE 9-1 Content of Registers in Pipeline Example

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	$R1$	$R2$	$R3$	$R4$	$R5$
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

- Pipelining does not reduce the time for a single task
- It increases the number of tasks completed per unit time (throughput)

Instruction Pipeline -Four-Segment

- Instruction pipeline divides instruction execution into different stages so that **multiple instructions are processed parallelly**, each at a different stage of execution.
- **Pipeline Segments:** The instruction cycle is divided into four segments:

- **FI (Fetch Instruction)**

  Fetches the instruction from memo

- **DA (Decode and Address)**

  Decodes the instruction

  Calculates the effective address

- **FO (Fetch Operand)**

  Fetches the required operand

- **EX (Execute)**

  Performs the operation

	Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	–	–	FI	DA	FO	EX			
	5					–	–	–	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

Fig: Timing of Instruction Pipeline

- **Working**:
  - Each instruction passes through all four stages
  - Different instructions are processed **simultaneously in different stages**
  - After initial delay, one instruction is completed in **every clock cycle**

RISC Pipeline

- RISC uses a **small and simple instruction set**.
- Each instruction is executed in **one clock cycle** using pipelining.
- The instruction cycle is divided into stages to allow **parallel execution**.
- Improves **throughput (one instruction per cycle)**.
- Consider there are **three-Segment Pipeline** stages as follows:

1) I – Instruction Fetch

- Fetch instruction from memory
- Decode instruction
- Select registers

2)A – ALU Operation

- Perform arithmetic/logic operations
- Calculate effective address
- Compute branch address

3)E – Execute

- Store result in register
- Perform memory read/write
- Update Program Counter

RISC Pipeline –Delay Load

- **Example:**
  1. LOAD: $R1 \leftarrow M[\text{address } 1]$
  2. LOAD: $R2 \leftarrow M[\text{address } 2]$
  3. ADD: $R3 \leftarrow R1 + R2$
  4. STORE: $M[\text{address } 3] \leftarrow R3$

- The three-Segment RISC Pipeline for the given example is shown in Fig (a) and Fig (b).

Delay Load:

- A load instruction needs time to get data from memory, so the next instruction may not get it immediately.
- Fig (a) shows pipelining without delay, ADD uses R1 and R2 before data is ready causes data hazard, which is called Delay Load.
- To avoid this data conflict, insert NOP (No Operation) before the add operation which is shown in Fig (b).

Clock cycles:	1	2	3	4	5	6
1. Load *R*1	I	A	E			
2. Load *R*2		I	A	E		
3. Add *R*1 + *R*2			I	A	E	
4. Store *R*3				I	A	E

(a) Pipeline timing with data conflict

Clock cycle:	1	2	3	4	5	6	7
1. Load *R*1	I	A	E				
2. Load *R*2		I	A	E			
3. No-operation			I	A	E		
4. Add *R*1 + *R*2				I	A	E	
5. Store *R*3					I	A	E

(b) Pipeline timing with delayed load

RISC Pipeline –Delay Branch

Delay Branch:

- In pipelining, a branch causes delay because the next instruction address is not immediately known.
- In delayed branch, the instruction is executed after the branch to avoid pipeline stall and improve performance.
- An example of delayed branch is shown in Fig (a) and Fig (b). The program for this example consists of five instructions:
  - Load from memory to R1
  - Increment R2
  - Add R3 to R4
  - Subtract R5 from R6
  - Branch to address X

Clock cycles:	1	2	3	4	5	6	7	8	9	10
1. Load	I	A	E							
2. Increment		I	A	E						
3. Add			I	A	E					
4. Subtract				I	A	E				
5. Branch to *X*					I	A	E			
6. No-operation						I	A	E		
7. No-operation							I	A	E	
8. Instruction in *X*								I	A	E

(a) Using no-operation instructions

Clock cycles:	1	2	3	4	5	6	7	8
1. Load	I	A	E					
2. Increment		I	A	E				
3. Branch to *X*			I	A	E			
4. Add				I	A	E		
5. Subtract					I	A	E	
6. Instruction in *X*						I	A	E

(b) Rearranging the instructions

ADITYA UNIVERSITY

Array Processor

- An **array processor** is a processor used to perform computations on **large arrays of data**.
- There are two types of Array Processors.
  1. Attached Array Processor
  2. SIMD Array Processor

Attached Array Processor

- An **array processor** is a processor used to perform computations on **large arrays of data**.
- One type of Array Processor is Attached Array Processor and its block diagram as shown in Fig.
- A general-purpose computer (host) controls the overall operation.
- The attached array processor is connected through an input–output interface and works as a co-processor to perform high-speed computations.
- The host sends data and instructions to the array processor via the interface.
- The array processor uses its own local memory for fast processing of large data arrays.
- A high-speed memory-to-memory bus connects the main memory and local memory, allowing quick data transfer.
- This setup improves performance by offloading complex numerical tasks from the host to the array processor.

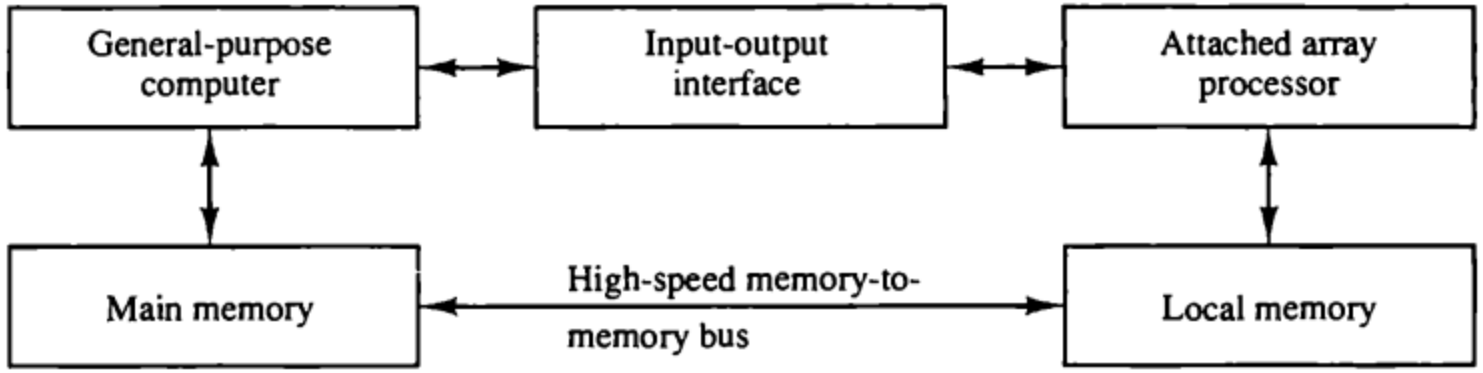


Fig: Attached Array Processor with host computer

SIMD Array Processor

- An **array processor** is a processor used to perform computations on **large arrays of data**.
- One type of Array Processor is SIMD Array Processor and its block diagram as shown in Fig.
- The Master Control Unit (MCU) controls the entire system and sends a single instruction to all processing elements.
- Multiple Processing Elements (PE_1, PE_2, …, PE_n) perform operations simultaneously on different data.
- Each processing element has its own local memory (M_1, M_2, …, M_n) to store data.
- All PEs receive the same instruction from the control unit but operate on different data values.
- The main memory supplies data and stores results.
- This organization follows SIMD (Single Instruction, Multiple Data) and is useful for vector and parallel processing.

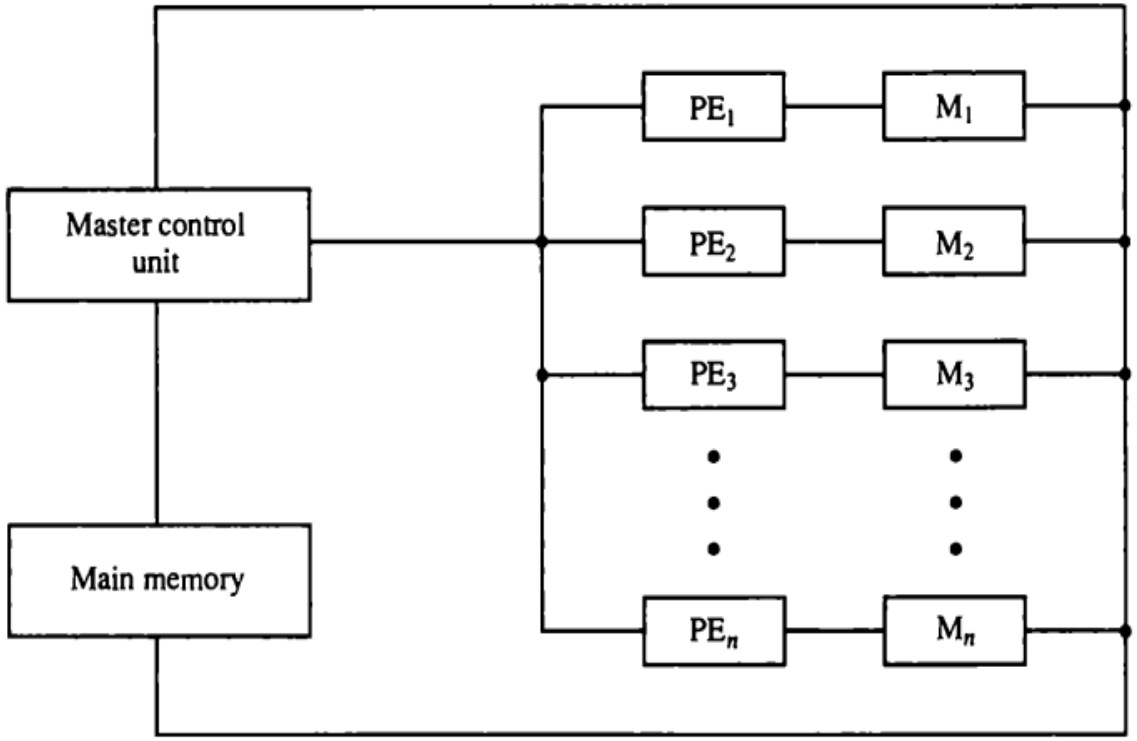


Fig: SIMD Array Processor with host computer